

Web Semântica

Experimentação / Resultados / Desempenho / Ablação /
Conclusão

Pedro Saito

Universidade Federal do Rio de Janeiro

2026-06-29



UFRJ

1. Experimentação

FedShop

Benchmark para medir a escalabilidade de engines SPARQL federadas¹.

Cada execução fixa uma combinação:

engine × **consulta** × **instância** × **batch**

O objetivo é isolar o impacto do tamanho da federação e da estratégia da engine.

Eixo	Configuração
Consultas	12 templates (q01–q12)
Instâncias	10 valores por template
Escala	2 batches: 20 e 40 endpoints
Fontes	vendedores + sites de avaliação
Engines	6 engines avaliados
Artefatos	results.csv, stats.csv, provenance.csv

1.2 Engines Avaliados

1. Experimentação

Engine	Estratégia principal	Linguagem	Execuções
FedShop-Go	ASK + cache + bound join	Go	68
FedX	ASK exclusivo + bound join	Java	69
RSA	SERVICE pré-atribuído (baseline)	Go	49
SPLENDID	VoID + estimativa de cardinalidade	Java	54
SemaGrow	Metadados de endpoint	Java	49
PyFedX	Protótipo Python (pyfedx)	Python	49

Dimensão	Métricas	Pergunta respondida
Correção	nb_results, mismatch	A resposta equivale à referência?
Tempo	exec_time, source_selection_time, planning_time	Onde o tempo é consumido?
Rede	ask, http_req, data_transfer	Quanto custa consultar a federação?
Seleção	tpwss, nb_distinct_sources, relevant_sources_selectivity	Fontes irrelevantes selecionadas?
Robustez	is_timeout, is_error	O plano é estável sob carga?

2. Resultados

Engine	Tempo Médio (s)	Execuções OK	Ranking
RSA	3,46	49 / 49	1º
FedShop-Go	7,19	43 / 68	2º
FedX	9,23	69 / 69	3º
PyFedX	10,57	26 / 49	4º
SPLENDID	19,22	54 / 54	5º
SemaGrow	85,53	49 / 49	6º

RSA é a baseline mais rápida por usar SERVICE pré-atribuído — sem seleção de fontes.

FedShop-Go é competitivo, mas acumula 25 timeouts em consultas com muitos intermediários.

100%

**FedX · RSA ·
SPLENDID ·
SemaGrow**

Nenhum timeout. Todos completaram todas as execuções previstas.

63,2%

FedShop-Go

25 timeouts em q02, q04 e q05 — consultas com grande volume de resultados intermediários.

53,1%

PyFedX

Timeouts em consultas com múltiplas fontes — protótipo sem otimizações de execução.

2.3 Desempenho por Consulta

Consulta	FedShop-Go	FedX	RSA	SPLENDID	SemaGrow	PyFedX
q01	5,62	120,07	6,78	10,17	93,23	T/O
q02	T/O	1,02	22,09	120,69	92,48	T/O
q03	5,03	1,59	2,36	14,33	1,72	T/O
q04	61,48	30,95	2,38	29,21	82,94	T/O
q05	T/O	1,09	1,83	97,29	120,04	T/O
q06	0,47	2,25	3,47	13,48	66,60	84,75
q07	0,68	0,84	2,15	167,18	120,07	7,37
q08	0,17	0,90	1,89	2,58	120,12	0,69
q09	0,02	1,40	1,82	0,59	63,64	0,05
q10	0,50	0,79	1,63	3,31	70,63	0,31
q11	0,14	1,39	1,76	1,28	92,62	0,10
q12	0,73	0,70	1,89	10,95	120,06	8,80

< 2s rápido 2–15s médio 15–60s lento > 60s / T/O crítico

FedShop-Go (batch 0, runs OK)

Métrica	Média
ASK probes	166 por execução
HTTP requests	366 por execução
Transferência	7,4 MB por execução
Fontes distintas	20 endpoints

ASK probes verificam cada triple pattern em cada endpoint — custo linear no número de padrões × endpoints.

Seleção de Fontes

Componente	Valor
source_selection_time	0,30s avg
Fração do exec_time	4,2%
planning_time	< 0,001ms
tpwss	166 (= # ASK)
relevant_sources_sel.	100% (batch 0)

Seleção de fontes consome apenas **4,2%** do tempo total — o gargalo está na execução dos joins.

3. Desempenho

Componente	Tempo Médio	% Total	Min	Max
Seleção de Fontes (source_selection_time)	0,30s	4,2%	0,01s	1,2s
Planejamento (planning_time)	< 0,001ms	≈ 0%	0s	0,001s
Execução + Joins (residual)	6,89s	95,8%	0,02s	120s
Total (exec_time)	7,19s	100%	0,02s	120s

Achado: a seleção de fontes via ASK é rápida. O custo dominante é o

Implicação: otimizações de join (bound join, reordenação de triple patterns) têm

3.1 Breakdown de Tempo

3. Desempenho

processamento de joins e o volume de dados trocado com os endpoints.

maior potencial de ganho do que acelerar os probes ASK.

Batch	Endpoints	Tempo Médio
batch_0	20	4,68s
batch_1	40	9,59s
Razão	×2	×2,05

Batch	Endpoints	Taxa de Sucesso
batch_0	20	66% (21/32)
batch_1	40	61% (22/36)

Comportamento observado:

O tempo de execução dobra ao dobrar o número de endpoints — escalabilidade **linear** com a federação.

A taxa de sucesso cai levemente (66% → 61%), sugerindo que consultas na fronteira do timeout tornam-se falhas com mais endpoints.

Contexto: apenas batches 0 e 1 foram avaliados neste trabalho. A escala

completa (até 200 endpoints) é trabalho futuro.

4. Estudo de Ablação

Variante	ASK	Cache	Bound Join	Metadados
Baseline (broadcast)	—	—	—	—
Seleção dinâmica	✓	—	—	—
Seleção reutilizada	✓	✓	—	—
Execução vinculada	✓	✓	✓	—
Plano completo	✓	✓	✓	✓

Controle experimental

Mesma consulta, instância, batch, tentativa e infraestrutura para todas as variantes.

Efeito esperado por componente

- **ASK**: eliminar fontes irrelevantes (-HTTP, -tempo)
- **Cache**: reutilizar seleção em instâncias similares

4.1 5 Variantes

- **Bound Join**: reduzir resultados intermediários
- **Metadados**: reordenar triple patterns por cardinalidade

4.2 Impacto por Variante

Seleção Dinâmica (ASK)

Reduz fontes irrelevantes — essencial para q06–q12 onde apenas 2–4 endpoints têm dados relevantes.

Efeito: menos HTTP requests e joins desnecessários.

Seleção Reutilizada (Cache)

Ao reaproveitar o mapeamento ASK entre instâncias similares, elimina 166 probes adicionais por consulta.

Efeito: reduz `source_selection_time` em consultas repetidas.

Bound Join + Metadados

Bound join envia bindings parciais ao endpoint, reduzindo resultados intermediários.

Metadados reordenam padrões.

Efeito: crítico para q02/q05 (ainda sem resolução).

A variante **Plano Completo** é o FedShop-Go avaliado neste trabalho. O estudo de ablação completo (rodando cada variante no FedShop) é trabalho futuro.

5. Conclusão e Trabalhos Futuros

Achado 1 — Queries seletivas: FedShop-Go supera FedX em q06–q11.

Em q09, 0,02s vs 1,40s do FedX — **70× mais rápido** em consultas de domínio único com fontes concentradas.

Achado 2 — Gargalo de joins: 95,8% do tempo é gasto em execução e joins.

A seleção de fontes (ASK) consome apenas 4,2% — o custo está em processar os resultados intermediários.

Achado 3 — Escalabilidade linear: dobrar os endpoints (batch 0→1) dobra o tempo de execução.

Consultas q02 e q05 geram timeout independente do batch — problema estrutural de cardinalidade.

Limitações atuais

- ✗ q02 e q05: timeout estrutural (cardinalidade)
 - ✗ 63,2% taxa de sucesso (25 timeouts)
- Somente batches 0 e 1 avaliados
- Ablação completa não executada no FedShop
- Sem métrica de correção (mismatch pendente)

Próximos passos

1. **Filter pushdown** — empurrar filtros para o endpoint antes de materializar resultados.
2. **Planner baseado em custo** — reordenar triple patterns por cardinalidade estimada (CostFed²).
3. **Resolver q02 / q05** — limitar intermediários com execução paginada.
4. **Escala completa** — executar batches 2–9 (até 200 endpoints).

5. **Ablação no FedShop** — comparar as 5 variantes com dados reais.

Referências

- (1) Dang, M.-H.; Aimonier-Davat, J.; Molli, P.; Hartig, O.; Skaf-Molli, H.; Crom, Y. L. FedShop: A Benchmark for Testing the Scalability of SPARQL Federation Engines. Em *The Semantic Web – ISWC 2023*; Lecture Notes in Computer Science; Springer, 2023; Vol. 14266, pp 285–301. https://doi.org/10.1007/978-3-031-47243-5_16.
- (2) Saleem, M.; Potocki, A.; Soru, T.; Hartig, O.; Ngomo, A.-C. N. CostFed: Cost-Based Query Optimization for SPARQL Endpoint Federation. Em *Proceedings of the 14th International Conference on Semantic Systems (SEMANTiCS 2018)*; Procedia Computer Science; Elsevier, 2018; Vol. 137, pp 163–174. <https://doi.org/10.1016/j.procs.2018.09.016>.
- (3) Acosta, M.; Vidal, M.-E.; Lampo, T.; Castillo, J.; Ruckhaus, E. ANAPSID: An Adaptive Query Processing Engine for SPARQL Endpoints. Em *The Semantic Web – ISWC 2011*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 18–34. https://doi.org/10.1007/978-3-642-25073-6_2.
- (4) Schwarte, A.; Haase, P.; Hose, K.; Schenkel, R.; Schmidt, M. FedX: Optimization Techniques for Federated Query Processing on Linked Data. Em *The Semantic Web – ISWC 2011*; Lecture Notes in Computer Science; Springer, 2011; Vol. 7031, pp 601–616. https://doi.org/10.1007/978-3-642-25073-6_38.
- (5) Görlitz, O.; Staab, S. SPLENDID: SPARQL Endpoint Federation Exploiting VOID Descriptions. Em *Proceedings of the Second International Workshop on Consuming Linked Data (COLD 2011)*; CEUR Workshop Proceedings; CEUR-WS.org, 2011; Vol. 782, pp 13–24.
- (6) Charalambidis, A.; Troumpoukis, A.; Konstantopoulos, S. SemaGrow: Optimizing Federated SPARQL Queries. Em *Proceedings of the 11th International Conference on Semantic Systems (SEMANTiCS 2015)*; ACM, 2015; pp 121–128. <https://doi.org/10.1145/2814864.2814886>.